

PRD · v2 (Python)

个性化可穿戴健康洞察平台

Distributed Wearable Health Insights Platform

产品描述 + 架构实现 PRD (全栈 Python 实现)

作者: Xueying Tang

日期: 2026年8月11日

状态: 草案 · 个人求职项目规划

目录

1. 产品概述.....	3
2. 目标与非目标	3
3. 核心功能需求	4
4. 系统架构.....	4
5. 非功能性需求	6
6. 评估与测试策略	7
7. 里程碑与时间线（8 周计划）	8
8. 成功指标（用于简历量化）	9
9. 风险与应对	9
10. 范围之外 / 后续可扩展方向.....	10

1. 产品概述

1.1 背景与问题陈述

可穿戴设备（智能手表、健身环、CGM 等）能持续产出高频生理信号，但大多数消费者产品只做到“展示原始数据”，缺乏两个关键能力：一是能在大规模并发设备场景下稳定、低延迟地处理数据流的分布式基础设施；二是能把原始信号转化为可信、可解释、经过评估验证的个性化健康建议的 AI 推理层。本项目旨在自己独立设计并实现一个覆盖这两个能力、并具备企业级交付质量（认证、权限、审计）的完整平台。

1.2 产品愿景

构建一个端到端的个性化健康洞察平台：从可穿戴设备的高频信号摄取开始，经过安全可回滚的特征提取管线，到基于大语言模型的、附带评估与防护机制的个性化建议生成，最终以企业级全栈应用的形式安全地交付给最终用户（患者）与协作角色（教练/医护人员）。

1.3 目标用户与角色

- 患者（Patient）：绑定可穿戴设备，查看个人健康洞察与历史趋势。
- 教练 / 医护协作角色（Coach）：在授权范围内查看所负责患者的聚合健康洞察，用于远程指导。
- （系统内部）平台运维角色：管理特征提取算法的版本发布与回滚，查看系统监控与审计日志。

1.4 本项目在求职故事线中的定位

本项目并非孤立的作品集项目，而是有意设计为串联现有三段经历的“旗舰项目”：数据接入与处理层借鉴在字节跳动学到的分布式系统设计模式（配置灰度发布、分布式追踪、高并发数据处理），但改用 Python（asyncio）实现，用于系统性精进个人 Python 工程能力，同时证明工程设计能力可以跨语言迁移，不绑定具体技术栈；AI 推理层呼应在 UNC 的 LLM 精准健康研究方向（但采用不同的生理信号类型与不同的工程重点，避免与研究项目内容重复）；全栈交付层对应在 BioCryst 的全栈平台开发经验。三层合一，证明可以独立完成从基础设施到产品交付的完整闭环。

2. 目标与非目标

2.1 目标（Goals）

- 验证系统能够以可接受的延迟处理来自大量并发模拟设备的高频生理信号（具体并发数与延迟目标见第 9 节，须以实测压测结果为准）。
- 实现特征提取算法的安全发布机制：版本校验、灰度发布、一键回滚。
- 实现附带评估框架（幻觉率、可溯源性）的 LLM 健康洞察生成服务，而非简单的 prompt 包装。

- 交付一个具备身份认证、基于角色的访问控制与审计日志的全栈应用，满足健康数据场景下的基本合规意识。
- 产出可用于技术求职的量化工程指标：吞吐量、延迟分位数、缓存命中率、评估准确率等，且均来自真实测试，不编造。
- 通过本项目系统性提升 Python 异步编程、性能调优与工程化能力，弥补此前 Python 经验偏应用层（FastAPI CRUD、脚本）、缺少高并发系统设计实践的短板。

2.2 非目标（Non-Goals）

- 不追求医疗级诊断准确性，所有生成的健康建议均为一般性生活方式参考，非临床诊断。
- 不对接真实医院系统或真实患者数据，全部使用公开研究数据集（如 WESAD、PPG-DaLiA）模拟。
- 不实现完整的 HIPAA 合规认证流程，仅在架构上体现审计日志、访问控制等合规相关工程实践。
- 不追求生产级高可用（如多可用区容灾），聚焦核心分布式与 AI 工程能力的展示。

3. 核心功能需求

3.1 数据接入与处理（Layer 1）

- 支持模拟设备通过异步 HTTP/WebSocket 持续上报心率、HRV、睡眠、活动量等生理信号。
- 信号先写入消息队列（Kafka 或 Redis Streams）进行削峰缓冲，再由特征提取服务消费。
- 特征提取服务基于 NumPy/SciPy 将原始信号窗口化处理为结构化特征（如静息心率、HRV 趋势、睡眠质量评分）。
- 特征提取算法支持多版本并存，可按比例灰度发布新版本，并在发现异常时一键回滚到上一稳定版本。
- 关键处理链路接入分布式追踪，可定位单条请求在各服务间的耗时与失败节点。
- 摄取服务基于 asyncio 构建，引入 uvloop 事件循环、连接池与批量写入优化，缓解 Python 相较编译型语言在原始吞吐量上的劣势。

3.2 AI 健康洞察生成（Layer 2）

- 消费结构化特征，调用 LLM 生成面向患者的个性化健康建议文本。
- 对相同或相似输入的请求结果进行缓存，避免重复调用带来的成本与延迟浪费。
- 维护一个人工构建的小型 benchmark 测试集，用于量化生成内容的幻觉率与可溯源性（是否有依据支撑建议内容）。
- 支持对不同 prompt 版本 / 模型版本进行 A/B 对比，记录各版本的评估分数、延迟与成本。

3.3 全栈交付平台（Layer 3）

- 患者可注册登录、绑定模拟设备、查看个人历史健康洞察与趋势图表。

- 教练角色可查看被授权访问的患者列表及其聚合洞察，用于远程指导场景。
- 所有身份认证基于 OAuth2 / JWT，接口按角色进行访问控制（RBAC）。
- 对健康数据的关键访问行为（谁在何时查看了谁的数据）写入审计日志，并提供查询接口。

4. 系统架构

4.1 架构总览

系统分为三层，自下而上分别是数据接入与处理层、AI 推理层、全栈交付层，横切面包括配置管理、可观测性与部署基础设施：

模拟可穿戴设备 → Python 异步摄取服务（asyncio + uvloop） → Kafka / Redis Streams → Python 特征提取服务（NumPy/SciPy，受配置管理服务控制的灰度发布）

特征提取服务的输出分两路：一路写入 TimescaleDB 做时序存储与历史查询；一路进入 LLM 推理服务（带缓存、评估、A/B 测试）生成个性化洞察。

两路结果最终都通过 FastAPI 后端 API 提供给 Next.js 前端仪表盘，后端统一处理鉴权（JWT）、权限控制（RBAC）与审计日志写入，数据落地于 PostgreSQL。

横切面：OpenTelemetry 负责全链路分布式追踪；Prometheus + Grafana 负责系统指标监报告警；GitHub Actions 负责持续集成与部署；Docker 负责各服务的容器化与本地/云端一致性。全部服务统一使用 Python 实现，便于集中精进异步编程与性能调优能力。

4.2 分层职责与关键设计决策

- 数据接入层改用 Python（asyncio + aiokafka）而非直接沿用在字节跳动使用的 Go 技术栈，核心目的是系统性精进异步并发编程与性能调优能力；事件驱动摄取、背压处理、批量写入等设计模式与 Go 版本思路一致，用以证明工程设计能力不绑定具体语言。
- 特征提取算法基于 NumPy/SciPy/Pandas 实现滑动窗口统计与频域特征计算，这是选择 Python 而非延续 Go 技术栈的另一项技术依据——Python 在科学计算与信号处理库的成熟度上具备明显优势，而不仅是个人技能精进需求。
- 配置管理服务参照字节跳动“验证-灰度-回滚”三段式发布流程设计，是本项目区别于普通“CRUD 项目”的核心亮点之一。
- AI 推理层刻意选用与 UNC 研究不同的生理信号（通用可穿戴信号而非 CGM 血糖），且工程重点放在 serving、缓存、评估与成本控制，而非模型微调本身，以确保与已有研究经历形成互补而非重复。
- 全栈交付层的 RBAC 与审计日志设计参考在 BioCryst 搭建 Microsoft Entra ID 权限体系的经验，但改用更轻量的 OAuth2/JWT 方案以适配独立开发的时间成本。
- 为弥补 Python 在原始吞吐量上相较 Go 的差距，摄取服务引入 uvloop 事件循环、连接池复用与批量写入优化，并将这一调优过程本身作为项目的技术亮点之一。

4.3 技术选型

层	技术	用途	对应经历
数据接入 / 处理层	Python (asyncio, aiokafka/confluent-kafka), uvloop	高频可穿戴信号的摄取、缓冲、并发处理	ByteDance 分布式设计模式迁移 + Python 工程能力精进
特征提取算法	NumPy, SciPy, Pandas	滑动窗口统计、频域分析等生理信号特征计算	Python 科学计算生态的天然优势
配置管理	自建版本化配置服务 (FastAPI + Pydantic, 校验/灰度/回滚)	特征提取算法的安全发布与回滚	ByteDance 配置管理经验 (模式迁移)
时序存储	TimescaleDB	存储原始信号与提取后的特征时间序列	新技术栈
AI 推理层	OpenAI API / 开源模型, Redis 缓存	生成个性化健康洞察, 缓存重复请求	UNC 研究 + BioCryst LLM 集成经验
后端 API	FastAPI	REST API、鉴权、业务逻辑	BioCryst 全栈经验
前端	Next.js	用户仪表盘、洞察可视化	BioCryst 全栈经验
数据库	PostgreSQL	用户账户、洞察历史、审计日志	BioCryst 全栈经验
认证与权限	OAuth2 / JWT, RBAC	患者 / 教练角色区分与访问控制	BioCryst RBAC 经验
可观测性	OpenTelemetry (Python SDK), Prometheus, Grafana	分布式追踪与系统监控	ByteDance 生产运维经验
压测	k6 / Locust	验证系统在并发负载下的表现	新技术栈
部署	Docker, GitHub Actions, Azure/AWS	容器化、CI/CD、云部署	BioCryst Azure 部署经验

4.4 核心数据模型

实体	关键字段	说明
User	id, email, role (patient/coach), created_at	平台用户账户, 区分患者与教练角色
Device	id, user_id, device_type, status	绑定到用户的模拟可穿戴设备
RawSignal	device_id, signal_type, value, timestamp	摄取层写入的原始高频生理信号
Feature	device_id, feature_type, value, window, algo_version	特征提取服务产出的结构化特征, 标注算法版本
ConfigVersion	id, algo_name, version, status, rollout_pct	特征提取算法的版本化配置, 支持灰度发布

实体	关键字段	说明
Insight	id, user_id, content, model_version, eval_score, created_at	LLM 生成的个性化健康建议及评估分数
AuditLog	id, actor_id, action, resource, timestamp	记录谁在什么时间访问/修改了哪些健康数据

4.5 关键 API 概览

方法	路径	说明
POST	/api/v1/devices/{id}/signals	摄取单条或批量原始生理信号（内部摄取服务调用）
GET	/api/v1/users/{id}/insights	获取某用户的历史健康洞察列表
POST	/api/v1/insights/generate	触发一次基于最新特征的个性化洞察生成
GET	/api/v1/features/{device_id}	查询某设备的结构化特征时间序列
POST	/api/v1/config/feature-algo	发布新版本的特征提取算法配置（灰度发布）
POST	/api/v1/config/feature-algo/{version}/rollback	回滚指定版本的算法配置
GET	/api/v1/audit-logs	（教练/管理员角色）查询审计日志
POST	/api/v1/auth/login	OAuth2 / JWT 登录鉴权

5. 非功能性需求

5.1 性能

系统需在压测中给出真实的以下指标（数值须在开发完成后通过 k6/Locust 实测得出，PRD 阶段暂不设定具体目标值，避免未验证前先设定不可信的承诺）：

- 持续处理的并发模拟设备数
- 端到端延迟 P50 / P95 / P99
- 信号摄取吞吐量（events/sec）
- LLM 推理服务的平均响应延迟与缓存命中率

5.2 可靠性与容错

- 消息队列消费失败需支持重试与死信队列，避免单条异常数据阻塞整条管线。
- 配置发布失败或异常需可在分钟级完成回滚。
- 关键服务需有健康检查与自动重启机制。

5.3 安全与合规意识

- 所有健康数据访问需经过身份认证与角色校验。
- 敏感操作（查看他人健康数据、发布配置变更）需写入审计日志，包含操作者、时间、对象。
- 不存储任何真实个人身份信息，所有测试数据均为公开研究数据集或合成数据。

5.4 可观测性

- 全链路分布式追踪覆盖从摄取到洞察生成的关键路径。
- Grafana 仪表盘展示系统吞吐量、延迟、错误率与资源使用情况。

6. 评估与测试策略

6.1 系统层面测试

- 单元测试（pytest）覆盖核心业务逻辑（特征提取算法、权限校验、审计日志写入等）。
- 集成测试验证摄取到洞察生成的端到端链路正确性。
- 使用 k6 / Locust 进行负载测试，模拟大量并发设备持续上报，记录延迟与吞吐量随负载变化的曲线。

6.2 AI 生成内容评估

- 构建小型人工标注 benchmark 集，覆盖典型健康信号模式与预期建议方向。
- 量化生成内容的幻觉率（是否包含无依据的医学声明）与可溯源性（是否能关联回具体输入特征）。
- 对比不同 prompt/模型版本在 benchmark 集上的评估分数、延迟与成本，作为版本迭代依据。

7. 里程碑与时间线（8 周计划）

总时间预算 1-2 个月。核心理程碑为第 4 周末达成端到端可演示链路，之后的时间用于逐层加深"工业级"证据（真实压测数据、监控、CI/CD）。全栈统一使用 Python 实现可省去多语言环境切换的成本，若个人可投入时间有限，可将整体计划顺延至 10-12 周，但应尽量保住第 4 周里程碑不大幅延后。

阶段	重点	交付物
第 0 周	准备	确定数据集（WESAD / PPG-DaLiA）、绘制架构图、搭建本地 docker-compose 环境、建立 repo 骨架
第 1-2 周	Layer 1 核心	Python 异步摄取服务（asyncio）+ Kafka/Redis Streams 接入 + 特征提取服务（NumPy/SciPy），跑通最简端到端链路
第 3 周	Layer 1 深化	配置管理服务（版本控制/灰度发布/回滚）+ OpenTelemetry 分布式追踪 + uvloop 性能调优

阶段	重点	交付物
第 4 周	Layer 2 核心（里程碑）	LLM 推理服务接入，消费结构化特征生成个性化建议；Redis 缓存；此时应有可演示的端到端链路
第 5 周	Layer 2 深化	评估框架（幻觉率/groundedness 打分、benchmark 集）+ prompt 版本 A/B 测试 + 成本/延迟追踪
第 6 周	Layer 3 核心	FastAPI 后端 API + PostgreSQL schema + JWT 认证 + RBAC + 审计日志
第 7 周	Layer 3 收尾	Next.js 前端仪表盘，接通后端，产出可用 demo 界面
第 8 周	打磨	k6/Locust 压测出真实并发/延迟/吞吐量数据 + Prometheus/Grafana 监控面板 + CI/CD + README 架构图 + demo 录屏

8. 成功指标（用于简历量化）

以下指标将在项目完成后基于真实测试结果填入简历 **Projects** 部分，PRD 阶段仅列出指标类型，不预设数值：

- 系统在压测中稳定支撑的并发模拟设备数
- 端到端处理延迟（P99）
- 信号摄取与处理吞吐量
- uvloop/批量写入优化带来的吞吐量提升比例（相较未优化版本）
- LLM 推理服务的缓存命中率带来的成本/延迟节省比例
- AI 生成内容在 benchmark 集上的评估准确率 / 幻觉率
- 单元测试覆盖率

9. 风险与应对

风险	影响	应对措施
全栈改用 Python，与 ByteDance 的 Go 技术栈直接关联减弱	面试讲"复刻 ByteDance 经验"的故事时说服力可能打折扣	强调迁移的是工程设计模式（灰度发布/分布式追踪/配置管理），而非语言本身；用 NumPy/SciPy 信号处理选型和 uvloop 性能调优作为新的技术亮点，正面回应"为什么用 Python"
个人时间有限，四层范围过大	项目可能烂尾或每层都做得很浅	保住第 4 周"端到端跑通"里程碑；若时间不够，优先做深 Layer 1，Layer 3 降级为最简 demo
AI 推理层与 UNC 研究项目内容重叠	简历上两个项目讲同一个故事，显得重复	使用心率/HRV/睡眠等通用可穿戴信号而非 CGM 血糖数据；明确本项目讲 serving/infra，UNC 项目讲模型本身

风险	影响	应对措施
缺乏真实设备数据，压测结果不可信	简历上的性能数字站不住脚，面试被追问会露馅	使用 WESAD/PPG-DaLiA 等公开真实生理数据集回放，而非随机生成假数据
LLM 调用成本失控	开发过程中 API 费用超预算	优先用小规模测试数据 + 缓存层；评估阶段用开源模型或限量测试
过早把未完成项目写上简历	面试被问细节答不上来，反而减分	仅在完成第 4 周里程碑、有可演示 demo 后再考虑写入简历，且所有数字必须来自实测

10. 范围之外 / 后续可扩展方向

- 接入真实可穿戴设备 SDK（如 Apple HealthKit、Fitbit API）替代模拟设备数据。
- 引入更严格的 HIPAA/GDPR 合规认证流程。
- 扩展多可用区部署与容灾能力。
- 探索联邦学习等隐私保护技术用于跨用户模型优化（如未来求职方向偏向 ML 研究，可作为独立扩展项目）。